# 操作系统作业二：ext4 文件系统

姓名：姚舜宇

学号：1190202107

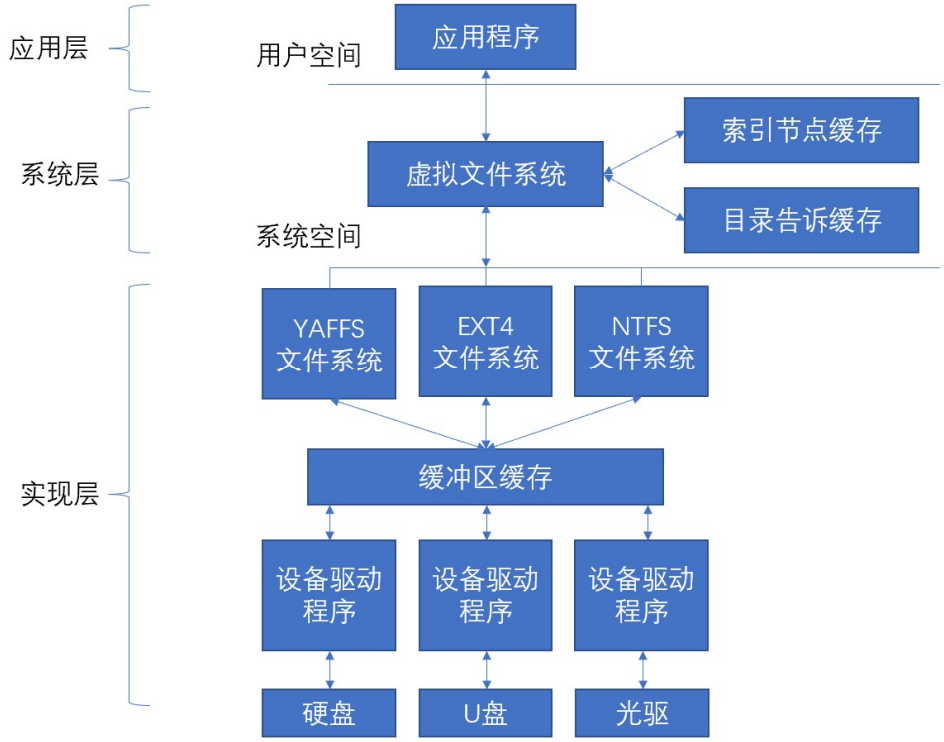
# 一. 文件系统

文件系统是操作系统用于明确存储设备或分区上的文件的方法和数据结构，即在存储设备上组织文件的方法。

操作系统中负责管理和存储文件信息的软件机构称为文件管理系统。

从系统角度看，文件系统是对文件存储设备的空间进行组织和分配，负责文件存储并对存入的文件进行保护和检索的系统。具体地说，它负责为用户建立文件，存入、读出、修改、转储文件，控制文件的存取，当用户不再使用时撤销文件等。

Linux 文件系统层次：




文件系统从大的方面可以分为两层：

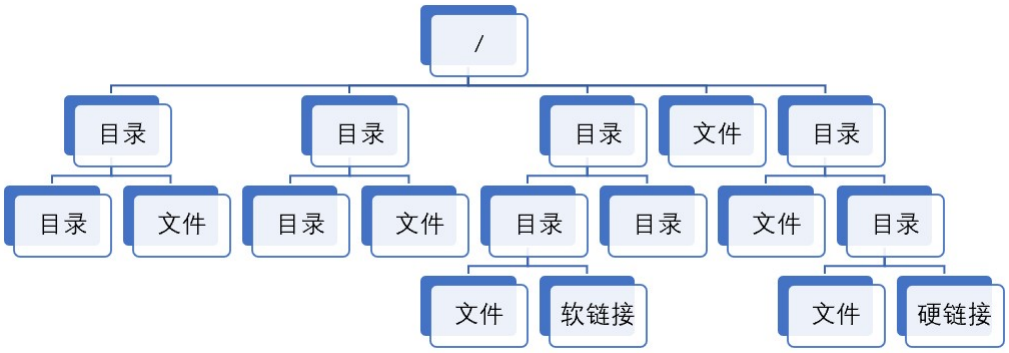
用户空间，如用户使用的各种应用程序；

内核空间，可以分为文件管理系统，包括虚拟文件系统和具体文件系统；设备驱动，包括缓冲区和设备驱动。

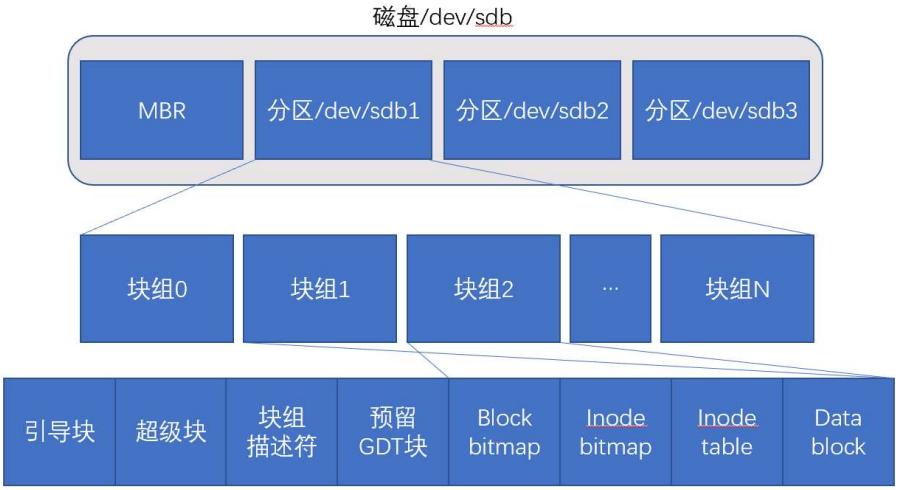
虚拟文件系统是对真实文件系统的抽象和软件实现，定义了所有文件系统都支持的、基本的、抽象的接口和数据结构。具体文件系统（如 ext4）实现了虚拟文件系统定义的抽象结构和数据结构，将自身的文件、目录等概念在形式上与虚拟文件系统的定义一致，在统一的接口和数据结构下隐藏了具体的实现细节。

# 二. Ext4 文件系统

Ext4 文件系统的布局：磁盘划分为分区，每个分区划分为 N 个块组。每个块组中，根据块组编号有不同的内部结构。下图分别简单描述了 ext4 文件系统的组织结构和布局。




文件系统的组织结构




Ext4 文件系统的布局

## 涉及到块组组成元素简介：

### 引导块：

该磁盘分区的第一个块，如果该分区安装了系统，则记录引导启动信息，用来引导启动该分区安装的系统。

### 超级块：

超级块记录关于封闭文件系统的各种信息，如块数量，inode 数量，支持的功能特性，维护信息等待。如果设置按照稀疏方式存储，则超级块和块组描述符的备份信息将只记录在组号码是 0 或者 3、5、7 的 n 幂的块组中。如果不设置，则备份信息将记录在所有块组中。一般只使用块组 0 中的超级块，因此也叫主超级块。如果主超级块损坏，可以从超级块的备份信息中复制数据来修复。

### 块组描述符（GDT）：

主要包含块位图，inode 位图和 inode 表位置，当前空闲块数，inode 数以及使用的目录数(用于平衡各个块组目录数)。每个块组都对应这样一个描述符。由于 GDT 对于定位文件系统的元数据非常重要，因此和超级块一样，也对其进行了备份。GDT 及其备份的内容都是一样的，所占块数也相同。

预留 GDT 块：

在创建文件系统时，为了便于将来扩展文件系统，在块组描述符表后面分配预留的数据块。

块位图（Block bitmap）：

Block Bitmap 是用来标识 Block Group 中的 block 使用情况，其中的每一位代表该 Block Group 中的一个 block，如果是 1 则代表使用，如果是 0 则代表空闲。通过 Block bitmap，可以快速知道该 block group 中每个 block 的使用情况。

Inode 位图（Inode bitmap）：

Inode 位图用于描述该块组所管理的 inode 的分配状态，如果 inode 位图中相应位置为 1，那么代表该 inode 已经分配出去；否则可以使用。

Inode 中包含的信息主要有：

Inode 编号；

文件的大小；

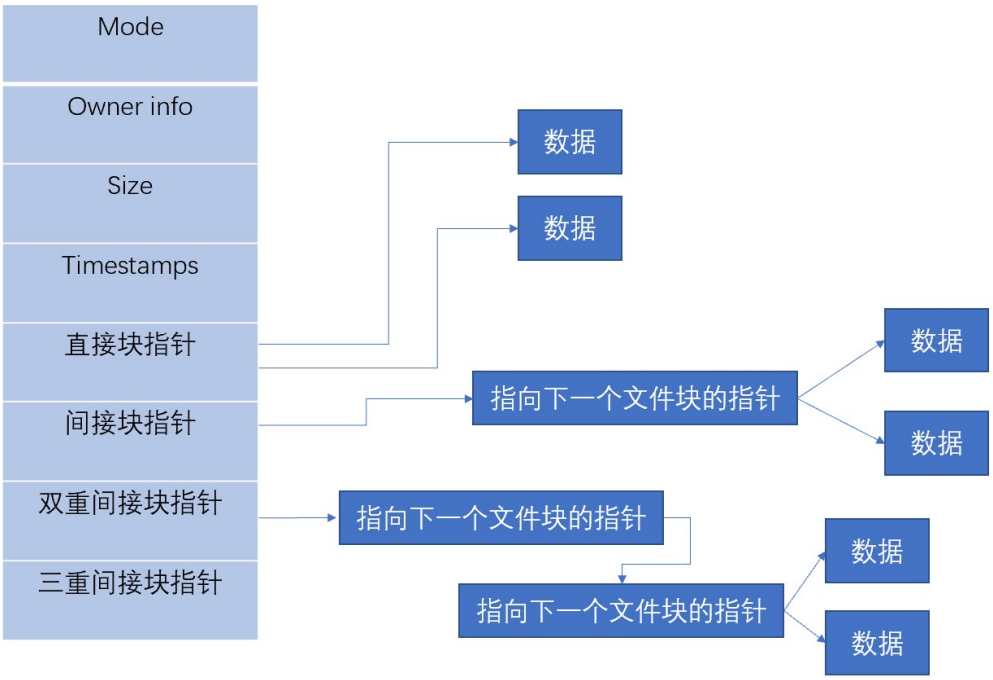
属主，属组，权限；

时间戳（三个，inode 上一次变动的时间，文件内容上一次变动的时间，文件上一次打开的时间）；

链接数；

指向存放文件实际数据的 block 的块指针。

Inode 结构如下所示。




Inode 表：

Inode 表用于存储 inode 信息。它占用一个或多个块(为了有效的利用空间，多个 inode 存储在一个块中)，其大小取决于文件系统创建时的参数，由于 inode 位图的限制，决定了其最大所占用的空间。

数据块：

上述元素存放了 ext4 文件系统的元数据，其他磁盘空间则用来存放文件真正的内容，因此称为数据块。

每个文件包含两部分，第一部分是我呢见的属性，如名称、大小、时间、权限等，它们被称为元数据，这部分数据是以 inode 的机构存储在数据块上；第二部分是文件内容，这部分内容直接存储在数据块上。

Inode 的结构体定义如下：

```
struct ext4_inode {
__le16  i_mode;     /* File mode */
__le16  i_uid;      /* Low 16 bits of Owner Uid */
__le32  i_size_lo;  /* Size in bytes */
__le32  i_atime;    /* Access time */
__le32  i_ctime;    /* Inode Change time */
__le32  i_mtime;    /* Modification time */
__le32  i_dtime;    /* Deletion Time */
__le16  i_gid;      /* Low 16 bits of Group Id */
__le16  i_links_count;  /* Links count */
__le32  i_blocks_lo;    /* Blocks count */
__le32  i_flags;    /* File flags */
union {
struct {
__le32  l_i_version;
} linux1;
struct {
__u32  h_i_translator;
} hurd1;
struct {
__u32  m_i_reserved1;
} masix1;
} osd1;             /* OS dependent 1 */
__le32  i_block[EXT4_N_BLOCKS];/* Pointers to blocks */
__le32  i_generation;   /* File version (for NFS) */
__le32  i_file_acl_lo;  /* File ACL */
__le32  i_size_high;
__le32  i_obso_faddr;   /* Obsoleted fragment address */
union {
struct {
__le16  l_i_blocks_high; /* were l_i_reserved1 */
__le16  l_i_file_acl_high;
__le16  l_i_uid_high;   /* these 2 fields */
__le16  l_i_gid_high;   /* were reserved2[0] */
__le16  l_i_checksum_lo;/* crc32c(uuid+inum+inode) LE */
```

```
__le16  l_i_reserved;
} linux2;
struct {
__le16  h_i_reserved1;  /* Obsoleted fragment number/size which are removed in ext4 */
__u16   h_i_mode_high;
__u16   h_i_uid_high;
__u16   h_i_gid_high;
__u32   h_i_author;
} hurd2;
struct {
__le16  h_i_reserved1;  /* Obsoleted fragment number/size which are removed in ext4 */
__le16  m_i_file_acl_high;
__u32   m_i_reserved2[2];
} masix2;
} osd2;             /* OS dependent 2 */
__le16  i_extra_isize;
__le16  i_checksum_hi;  /* crc32c(uuid+inum+inode) BE */
__le32  i_ctime_extra;  /* extra Change time      (nsec << 2 | epoch) */
__le32  i_mtime_extra;  /* extra Modification time(nsec << 2 | epoch) */
__le32  i_atime_extra;  /* extra Access time      (nsec << 2 | epoch) */
__le32  i_crtime;       /* File Creation time */
__le32  i_crtime_extra; /* extra FileCreationtime (nsec << 2 | epoch) */
__le32  i_version_hi;   /* high 32 bits for 64-bit version */
__le32  i_projid;   /* Project ID */
};
```

# 三. Ext4 文件管理

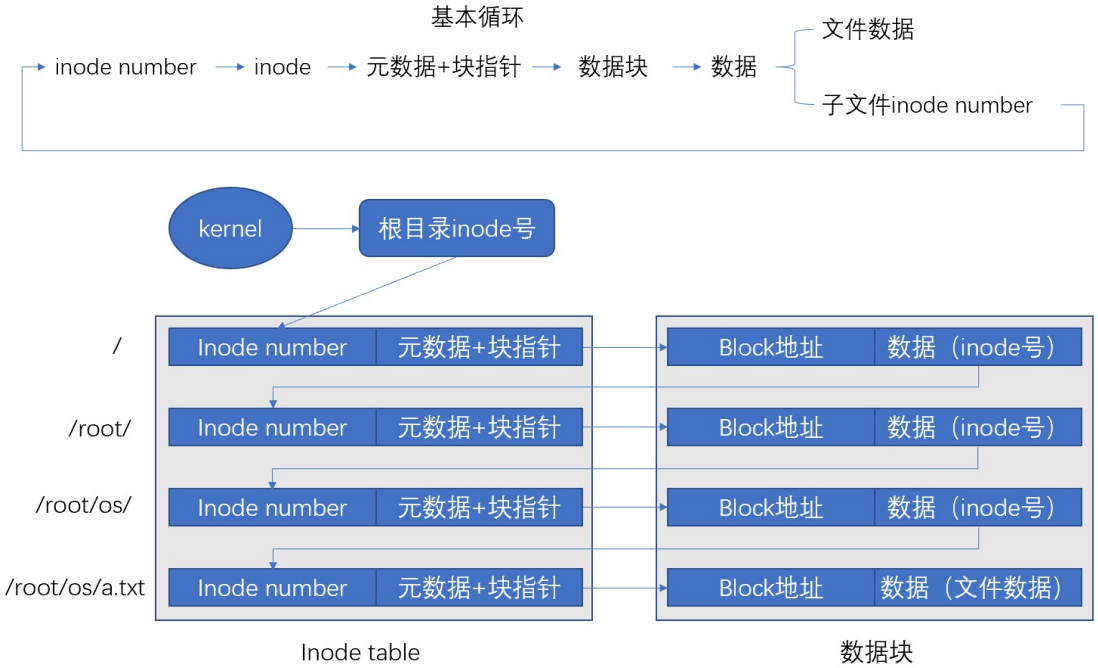
Ext4 文件系统内部使用 inode number 来识别文件。任何文件都有其分区内唯一的 inode number，并通过块指针与文件数据关联。文件名是相当于人为设定的名称，作用是方便人类记忆和使用。

Linux 系统认为一切都是文件，包括目录。目录文件的结构是一系列目录项的列表，每个目录项由两部分组成，目录直接子文件的文件名以及对应的 inode 号。根目录/由 Linux 内核直接管理并记录其 inode number。因此，我们可以通过根目录及其子目录，找到文件名对应的 inode number，再根据 indoe 中指向数据块的块指针，找到文件内容数据。

## 1. 查找文件

例如，查找文件的路径是/root/os/a.txt，步骤如下：

通过内核找到根目录/的 inode number；
根据根目录/的 inode number，到 inode table 中找到根目录/的块指针；
通过块指针找到根目录/的数据块，根据其中的数据找到下级目录/root/对应的 inode number；
根据/root/对应的 inode number，到 inode table 中找到目录/root/的元数据和块指针；
通过块指针找到存放目录/root/的数据块，根据其中数据找到/root/os/目录对应的 inode number；
根据/root/os/目录对应的 inode number，到 inode table 中找到目录/root/os/的元数据及块指针；
通过块指针找到存放目录/root/os/的数据块，根据其中的数据找到文件/root/os/a.txt 对应的 inode number；
根据文件/root/os/a.txt 对应的 inode number，到 inode table 中找到文件/root/os/a.txt 的元数据及块指针；
通过块指针找到存放文件/root/os/a.txt 的数据块，完成查找。
大致流程图如下：




2. 创建文件

例如，需要创建文件/root/os/a.txt，步骤如下：
扫描 inode bitmap，找到一个空闲 inode，将其标记为占用，获取对应的 inode number；
查找目录/root/os/的数据，并在其中添加一条目录项记录，文件名为/root/os/a.txt；
根据 inode number，在 inode table 中找到改 inode，记录文件/root/os/a.txt 的部分元数据，如 inode number、权限等；
扫描 block bitmap，找到可分配的空闲块，标记为占用，并在文件/root/os/a.txt 的元数据中记录块指针；
在数据块中写入文件数据。

3. 删除文件

例如，需要删除文件/root/os/a.txt，步骤如下：

查找文件/root/os/a.txt 使用的元数据信息；
查找目录/root/os/的数据，并删除文件/root/os/a.txt 的目录项记录；
修改文件/root/os.a.txt 的元数据，将其链接数减一；
如果此时文件/root/os/a.txt 的链接数不大于 0，扫描 block bitmap，将分配给文件的块节点标记为空闲；
如果此时文件/root/os/a.txt 的链接数不大于 0，扫描 block bitmap，将其中对应的节点标记为空闲。